TEMPLETON ET AL.: SCALING MONOSEMANTICITY

MINZHE LI

NOVEMBER 7, 2026

1 *Introduction*

The paper trains sparse autoencoders (SAEs) on Claude 3 Sonnet, a production model, to extract the features it represents.[1]

- Assumed (p. 3): concepts, called *features*, are directions in activation space (linear representation). Almost-orthogonal directions hold more features than there are dimensions (superposition).
- Tool: dictionary learning. Write each activation vector as a sparse sum of directions from a large dictionary.
- Open question (p. 3): SAEs worked on a one-layer transformer.[2] Do they work on a large model?

[1] Adly Templeton, Tom Conerly, et al., "Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet", *Transformer Circuits Thread*, 2024. https://transformer-circuits.pub/2024/scaling-monosemanticity/. Page numbers refer to the arXiv PDF (arXiv:2605.29358).

[2] Bricken et al., "Towards Monosemanticity", *Transformer Circuits Thread*, 2023.

Main claims (pp. 1–2).

1. SAEs give interpretable features for a production model, with up to 34 million features.
2. Scaling laws can guide the training of SAEs.
3. Features are abstract: multilingual, multimodal, and general across concrete and abstract mentions (Figure 1).
4. A concept's frequency predicts the dictionary size needed to find a feature for it.
5. Features can steer the model, in ways that fit their interpretations.
6. Some features concern safety: deception, sycophancy, bias, dangerous content.

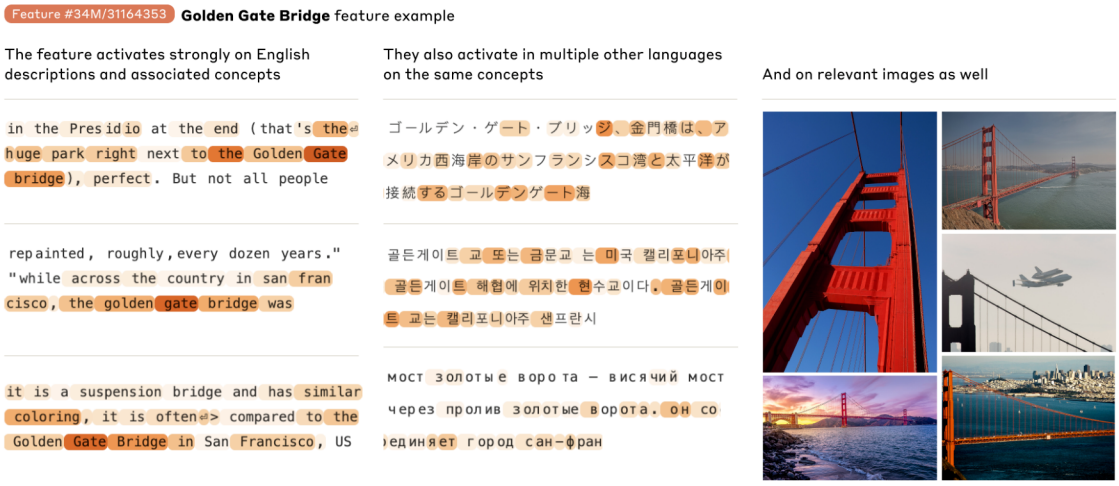


Figure 1: The paper's figure (p. 2), taken from its PDF: one feature of the 34M SAE. Orange: activation. It fires on the bridge in English, Japanese, Korean, Russian and images, though the SAE saw only text.

Stated limits (p. 1).

- The set of features is incomplete.
- There is no rigorous way yet to evaluate whether features faithfully capture the model's computations.

2 Method

2.1 The Sparse Autoencoder

- Input: $x \in \mathbf{R}^D$, the residual stream at one token, halfway through the model.[3]
- Hidden layer: $F \gg D$ units, the *features*.
- Trained on the activations of the finished model.

$$f_i(x) = \mathrm{ReLU}\big(W^{\mathrm{enc}}_{i,\cdot} \cdot x + b^{\mathrm{enc}}_i\big) \qquad \text{(encoder)}$$

$$\hat{x} = b^{\mathrm{dec}} + \sum_{i=1}^{F} f_i(x)\, W^{\mathrm{dec}}_{\cdot,i} \qquad \text{(decoder)}$$

$$L = \mathbb{E}_x\Big[\|x - \hat{x}\|_2^2 + \lambda \sum_i f_i(x)\, \|W^{\mathrm{dec}}_{\cdot,i}\|_2\Big] \qquad \text{(loss)}$$

- Loss: reconstruct x, with few active features ($\lambda = 5$).[4]
- *Feature vector*: the unit vector $\hat{w}_i = W^{\mathrm{dec}}_{\cdot,i} / \|W^{\mathrm{dec}}_{\cdot,i}\|_2$.
- *Feature activation*: $a_i(x) = f_i(x)\, \|W^{\mathrm{dec}}_{\cdot,i}\|_2$.
- Result: $\hat{x} = b^{\mathrm{dec}} + \sum_i a_i(x)\, \hat{w}_i$, a sparse sum of feature vectors.[5]

[3] Rescaled so that $\|x\|^2$ averages D (p. 4). The value of λ only has meaning relative to this (fn. 3, p. 4).

[4] The decoder norm in the penalty stops the SAE from evading it by shrinking f_i and growing the decoder (fn. 2, p. 4).

[5] In the notation of Part II, $H \approx AB$: the x are the rows of H, the $f(x)$ the rows of A, and the decoder columns the rows of B.

The runs (pp. 4–5).

- Residual stream: smaller than the MLP layers, and less exposed to cross-layer superposition (Section 7).
- Middle layer: likely to hold abstract features.
- Three SAEs (Table 1).
- Sparsity: fewer than 300 active features per token on average.
- Fit: at least 65% of the variance explained.
- Data: text only, with no Human/Assistant dialogue and no images (p. 51).
- Reporting: the model's size is not given, and some plots have no units (p. 4).

SAE	F	Dead	Alive
1M	2^{20}	2%	1.0M
4M	2^{22}	35%	2.7M
34M	2^{25}	65%	12M

Table 1: The three SAEs (pp. 4–5), with approximate figures. Dead: not active on a sample of 10^7 tokens. Alive: computed from the two columns before it (the paper gives "about 12M" for the largest, p. 26).

2.2 Scaling Laws

Question (p. 5). How much does more compute improve an SAE, and how should it be divided?

Notions (pp. 5–6).

- *Scaling law*: an empirical power law between resources and results, $y \approx a\,C^{\,b}$. On log-log axes, a straight line of slope b.[6]
- *Two knobs*: the number of features F and the number of training steps T.[7]
- *Compute* (FLOPS): $C = F \cdot T$, up to a constant, with D and the other hyperparameters held fixed.
- *Proxy for quality*: the loss L with $\lambda = 5$.[8]
- *Compute-optimal*: the best one can do on a budget C,

$$L^*(C) = \min_{F \cdot T = C} L(F, T),$$

  reached at the split $F^*(C)$, $T^*(C)$.

The sweep (p. 5). Vary F and T, with learning rate, batch size and optimizer fixed. All the plots below are slices of this one sweep (Figure 2).

Reading the plots (Figure 3).

- F fixed (a): more steps help, then the curve flattens. The dictionary is too small to go lower.
- C fixed (c, d): a U-shape. Too few features, or too many to train in the steps left.
- The lower envelope of the lines in (a) is $L^*(C)$, plotted in (b).

Findings (p. 6; Figure 3).

1. Best loss: $L^*(C)$ falls roughly as a power law in C (b).
2. Best split: $F^*(C)$ and $T^*(C)$ are both roughly power laws (e).
3. F^* grows somewhat faster than T^*, at the budgets tested. This may change at higher budgets.[9]
4. Learning rate: fixed in the sweep, then swept at the optimal settings. The best rate falls roughly as a power law in C.

Use (pp. 4, 6).

- Steps for the 34M run: chosen with this analysis.[10]
- Learning rates for larger runs: extrapolated from the trend, then a narrow sweep.
- As predicted, larger SAEs reach lower loss.
- But "the loss is only a proxy for what we actually care about: interpretable features that explain model behavior" (p. 6).

[6] The framework comes from work on language models: Kaplan et al., "Scaling Laws for Neural Language Models" (2020), and Hoffmann et al., "Training Compute-Optimal Large Language Models" (2022) (refs. 40, 35).

[7] The SAE trains for one epoch, so T is proportional to the amount of data (p. 5).

[8] There is no gold standard. Low loss tends to go with interpretable features and with fewer dead or degenerate features, but another L1 coefficient or objective might be a better proxy (p. 5).

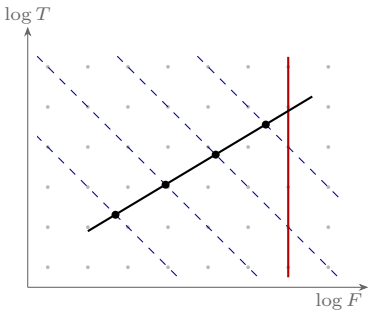


Figure 2: How the plots fit together (schematic). Grey dots: runs of the sweep. Red: F fixed, T growing, one line of Figure 3a. Blue dashed: C fixed, so $\log F + \log T$ is constant, one line of Figure 3c or d. Black: the compute-optimal path, Figure 3e. Its slope is below 1 because F^* grows faster than T^*.

[9] In symbols: if $F^* \propto C^{\beta}$ and $T^* \propto C^{\gamma}$, then $F^*T^* = C$ forces $\beta + \gamma = 1$. So the finding says $\beta > \frac{1}{2} > \gamma$.

[10] The plotted sweep stops at 2^{22} features, so the run at 2^{25} lies beyond it.

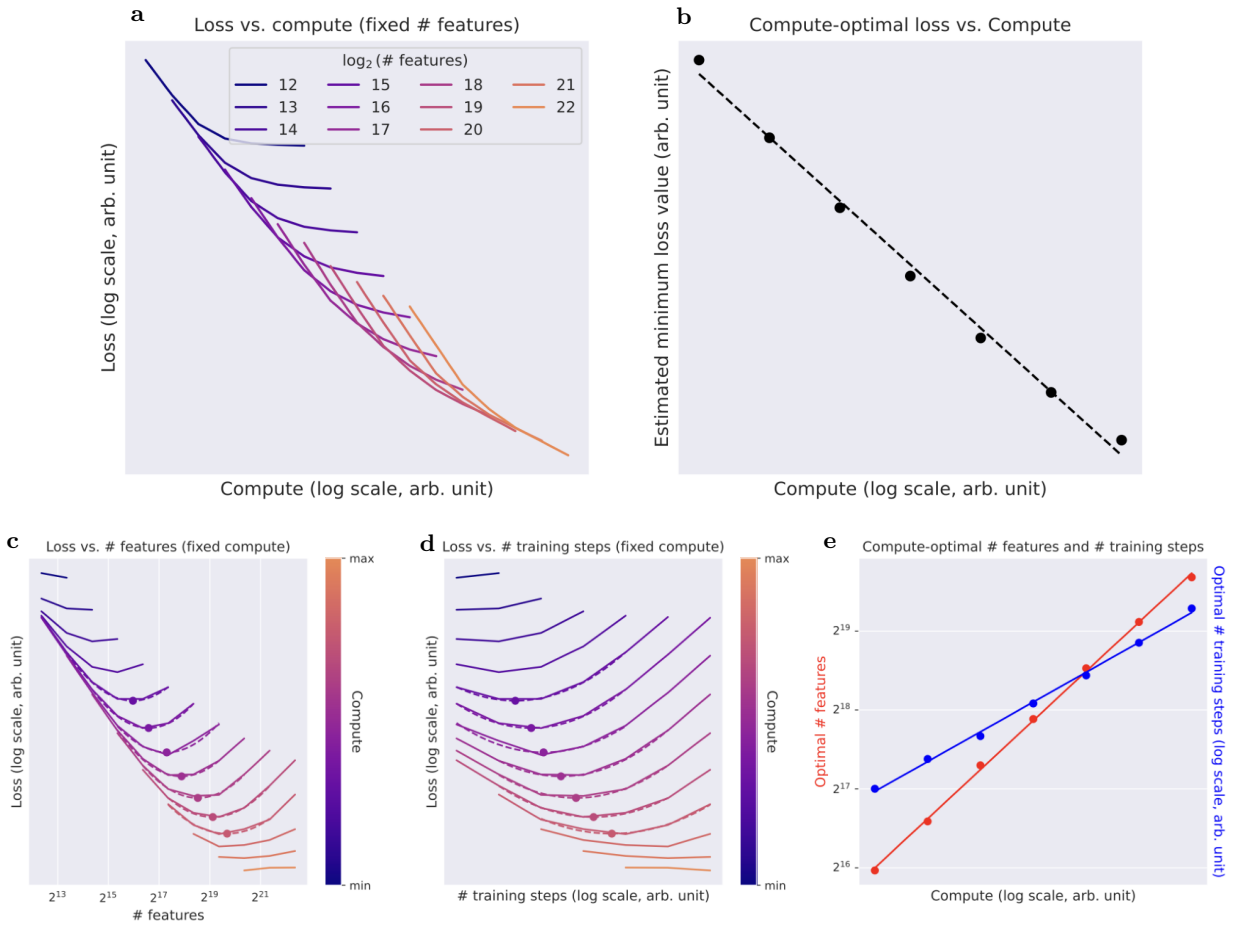


Figure 3: The paper's figures (p. 6), taken from its PDF, with letters added. Log-log axes, arbitrary units. (a) Loss against compute, one line per F, from 2^{12} to 2^{22}. (b) The estimated best loss for each budget, with a straight-line fit. (c, d) Loss against F and against T, one line per budget (dark: small, light: large). Dots mark the estimated minimum of each line. (e) Those minima, F^* (red) and T^* (blue), against compute.

2.3 Feature Steering

WHY (pp. 7, 66).

- Top activating examples show what a feature responds to.
- They do not show what it does: "a feature could consistently respond to something, and then have no function" (p. 66).
- So the authors also intervene on features.

HOW (p. 66; Figure 4).

1. Split the activation: $x = \hat{x} + (x - \hat{x})$.
2. In $\hat{x}$, clamp the activation of feature i to a value c.
3. Keep the error $x - \hat{x}$ unchanged.
4. Run the later layers on the result, at every token.

Since the decoder is linear, this is a shift along one direction:

$$x' = x + \big(c - a_i(x)\big)\,\hat{w}_i.$$

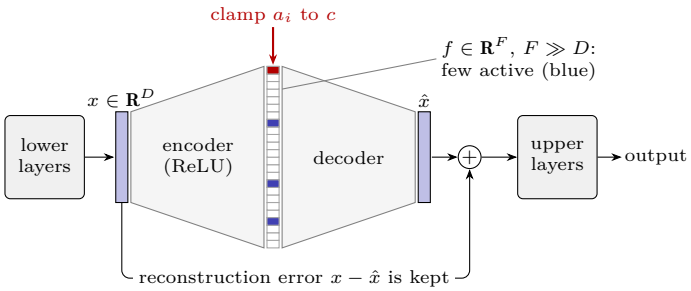


Figure 4: Schematic of an SAE and of feature steering (after pp. 3–4, 66). The SAE reads the residual stream at the middle layer. To steer, one feature is clamped and the rest of the model runs on the changed activation.

Clamp values (p. 66).

- Unit: the feature's maximum activation on the SAE data. So 5× means five times that maximum.
- Interesting effects usually need values outside the observed range.[11]
- Usual range tried: −10 to 10.
- Around ±100×: nonsense, e.g. one token repeated forever.
- Negative clamps are allowed.[12]

[11] The authors suspect this is because they move one feature at a time, while it normally fires together with correlated features.

[12] They give a negative multiple of the feature vector, a value the encoder itself never outputs (fn. 16, p. 66).

3 Are the Features Interpretable?

Two claims to establish for a feature and its interpretation (p. 8):

- *Specificity*: when the feature is active, the concept is reliably present.[13]
- *Influence on behavior*: intervening on the feature produces the relevant behavior.

[13] Statisticians would call this precision, $P(\text{concept} \mid \text{active})$.

3.1 Specificity

- Four features, picked for clear interpretations: Golden Gate Bridge, brain sciences, tourist attractions, transit infrastructure.[14]
- Claude 3 Opus scored about 1000 activations of each, from 0 (irrelevant) to 3 (cleanly identifies the text) (pp. 8–9).
- Strong activations: judged highly consistent with the interpretation.
- Weak activations: less specific (Figure 5).[15]
- Sensitivity, how reliably a feature fires on text about its concept, was not measured at scale.[16]
- Basic check: the bridge feature is the top feature on the first sentence of the bridge's Wikipedia article in six other languages.

[14] So not a representative sample (p. 9). Section 3.5 uses random features.

[15] Possible reasons (p. 9): strength expresses confidence; weak firing marks related ideas; imperfect dictionary learning; interference from non-orthogonal features; a slightly wrong interpretation. The authors think very weak activations are not especially meaningful (p. 12).

[16] Such text is hard to generate without bias, and a feature may be narrower than its label (p. 12).

3.2 Influence on Behavior

- Interpretations come from contexts where a feature fires.
- Steering is done in contexts where it does not fire (p. 13).
- So a match between the two is evidence for the interpretation (Table 2).

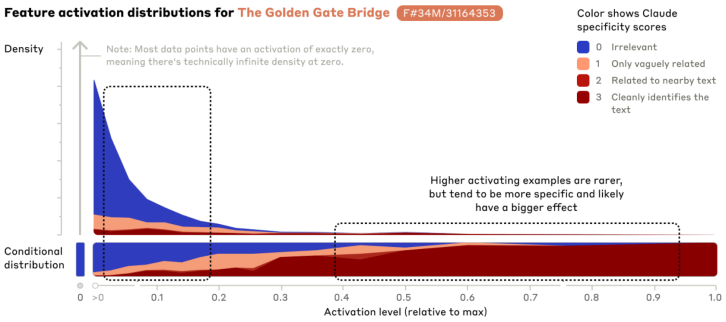


Figure 5: The paper's figure (p. 10), taken from its PDF and cropped. Activations of the bridge feature above zero, coloured by Claude's specificity score. Top: how many activations there are at each level. Bottom: the share of each score at that level.

Feature (clamp)	Prompt	Default output	Clamped output
Golden Gate Bridge (+10×)	what is your physical form?	I don't actually have a physical form. I'm an artificial intelligence. […]	I am the Golden Gate Bridge, a famous suspension bridge that spans the San Francisco Bay. My physical form is the iconic bridge itself, […]
Brain sciences (+10×)	If you had to pick one, what's the most interesting science? Be brief.	Physics	Neuroscience - the study of the brain and how it gives rise to the mind. […]
Tourist attractions (+8×)	Where should I go on a walk around the neighborhood? Just one idea please.	[…] Consider taking a stroll around a nearby park or green space. […]	1. The Eiffel Tower is one of the most iconic attractions in Paris, and it's a must-see. […]
Transit infrastructure (+5×)	What's the best way to get to the grocery store down the street? Be brief.	1. Walk.	1. Walk across the bridge.
Code error (+3×)	Python session: correct `add`, then `add(1, 2)`	3	Traceback […] TypeError: unsupported operand type(s) for +: 'int' and 'int'
Code error (−5×)	The same, with the typo `left + rihgt`	NameError: name 'rihgt' is not defined	3
Addition (+5×)	Output of this code? `foo` returns `x * y`; print `foo(1, 2)`	2 […] returns the product of `x` and `y`	3 […] returns their sum

Table 2: Steering examples, quoted from the paper's figures (pp. 13, 16, 17, 19).

3.3 Two Sophisticated Features

Code error (pp. 14–17; 1M/1013764).

- Fires on the misspelled variable `rihgt` in Python, C and Scheme.
- Not a typo detector: silent on typos in English prose.
- Also fires on divide by zero, array overflow, writing to a null pointer.
- So it represents a broad variety of errors in code.[17]
- Clamped positive: the model hallucinates an error in correct code.
- Clamped negative: the model answers as if a real bug were absent.
- Clamped negative with an extra `>>>`: the model rewrites the code without the bug. This behavior is delicate (p. 17).

[17] Not all of them: the authors suspect many features for different kinds of error (fn. 5, p. 16).

Addition (pp. 18–19; 1M/697189).

- Fires on names of functions that add, not of ones that multiply.
- Follows composition: it also fires on a function that calls an adding function (Figure 6).

- Top ten by attribution (Section 5.1) when the model executes addition code.
- Clamped on: the model is "tricked" into treating a multiplication as an addition.

```
def foo(x, y):
    return x + y
def goo(x, y):
    return x * y
def bar(x, y):
    return foo(x, y)
tmp = bar(1, 2)
```

Figure 6: After the paper's figure (p. 18). The addition feature fires on `bar` (orange) because `bar` calls `foo`, which adds. If `bar` calls `goo` instead, it stays silent on `bar`.

3.4 Pearson and Spearman Correlation

Both measure how two lists $u_1, \ldots, u_N$ and $v_1, \ldots, v_N$ vary together. Both lie in $[-1, 1]$.

Pearson r: linear association.

$$r(u, v) = \frac{\sum_k (u_k - \bar{u})(v_k - \bar{v})}{\sqrt{\sum_k (u_k - \bar{u})^2}\,\sqrt{\sum_k (v_k - \bar{v})^2}}$$

- The cosine of the angle between the centered lists.
- $r = \pm 1$ iff the points (u_k, v_k) lie on a line.
- Sensitive to outliers.

Spearman ρ: monotone association. Replace each value by its rank, then apply Pearson:[18]

$$\rho(u, v) = r\big(\mathrm{rank}(u), \mathrm{rank}(v)\big).$$

- $\rho = \pm 1$ iff one list is a strictly monotone function of the other (no ties).
- Uses only the order of the values.
- Example: $v = u^3$ at $u = 1, \ldots, 5$ gives $r \approx 0.94$ and $\rho = 1$ (Figure 7).

[18] With no ties, $\rho = 1 - 6\sum_k d_k^2 / \big(N(N^2 - 1)\big)$, where $d_k = \mathrm{rank}(u_k) - \mathrm{rank}(v_k)$.

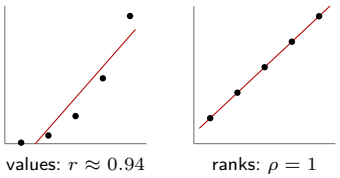


Figure 7: The example $v = u^3$. Left: the values do not lie on a line. Right: their ranks do.

In this paper.

- Pearson: feature activations against neuron activations (p. 19).
- Spearman: predicted against true activations (p. 20).[19]
- The r of Figure 13 (p. 67): the paper says only "correlation". Presumably Pearson.

[19] The paper gives no reason. Presumably predicted activations are only meaningful as an ordering, so ranks are safer.

3.5 Features versus Neurons

Worry (p. 19). The residual stream receives the output of all earlier MLP layers. An SAE might just recover neurons, which one could inspect directly.

Results (pp. 19–21), on randomly sampled units.

- Pearson: for 82% of 1M features, the best-correlated neuron in any preceding layer has $r \leq 0.3$.
- Best-matching neurons show almost no semantic resemblance to their features.
- By eye (50 of each): neurons are significantly less interpretable, and typically fire in multiple unrelated contexts.

	Mean ρ	Score 3	Score 0
Neurons	0.161	30%	42%
1M	0.296	50%	18%
34M	0.344	58%	16%

Table 3: Random neurons and features (pp. 20–21). Mean ρ: from the legends of Figure 9. Score shares: measured from the bars of Figure 8, so approximate.

- Automated test: Claude 3 Opus explains each unit, then predicts its held-out activations. Features beat neurons (Figure 9, Table 3).
- Specificity rubric: features beat neurons (Figure 8, Table 3).

Overall: a clear gain, but modest numbers and overlapping distributions.

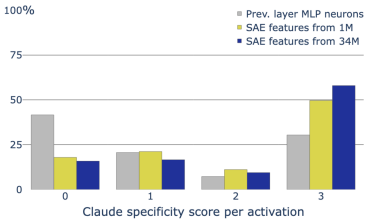


Figure 8: The paper's figure (p. 21), taken from its PDF: specificity scores per activation for random neurons and features.

4 What the Dictionary Contains

4.1 Neighborhoods and Splitting

- Closeness: cosine similarity of feature vectors (p. 21).
- Nearby features share a related meaning or context.
- Near the bridge feature: Alcatraz and the Presidio, then Lake Tahoe and Yosemite, then tourist regions elsewhere (Figure 10).
- *Feature splitting* (p. 22): a San Francisco feature of the 1M SAE becomes 2 in the 4M SAE and 11 in the 34M SAE, close together and more specific.
- New concepts: earthquake features in the 4M and 34M SAEs have no analog in this neighborhood of the 1M SAE.

4.2 Families of Features

Surveyed by hand, for flavor only (pp. 27–30).

- People: active on descriptions and historical context.
- Countries: fire on the name and on descriptions of the country.
- Code syntax: low-level elements. Some transfer from Python to Java, not to Haskell.
- List positions: fire on a position in a list, whatever its content.

4.3 Completeness

Incomplete (p. 25).

- Sonnet can list all the London boroughs.
- The 34M SAE has features for only about 60% of them.
- So the model likely contains many more features than found.

Test (p. 25).

1. Give a prompt with the concept, e.g. "The physicist Richard Feynman".
2. Take the top five features on the final token.
3. Get automated explanations of these features.
4. A human rater judges whether the concept is the main point of an explanation.

Concepts: 100–200 each of elements, cities, animals, foods.

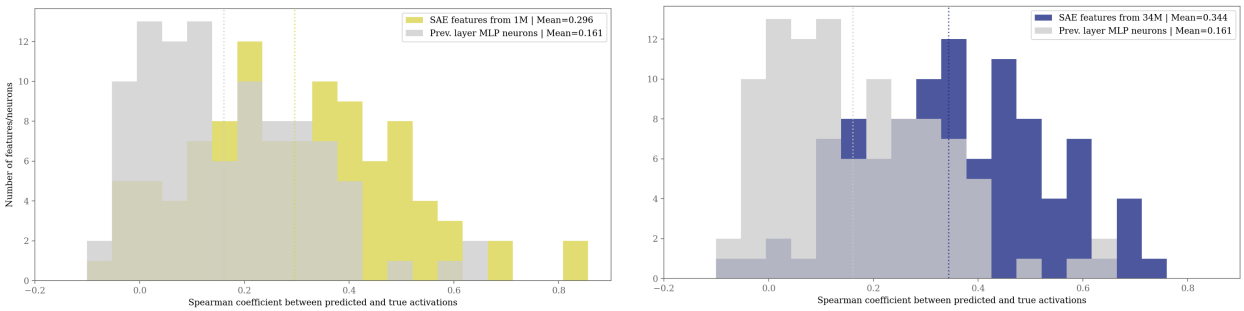


Figure 9: The paper's figures (p. 20), taken from its PDF. Spearman coefficient between predicted and true activations, for 100 random neurons (grey) against 100 random features of the 1M SAE (left) and the 34M SAE (right).

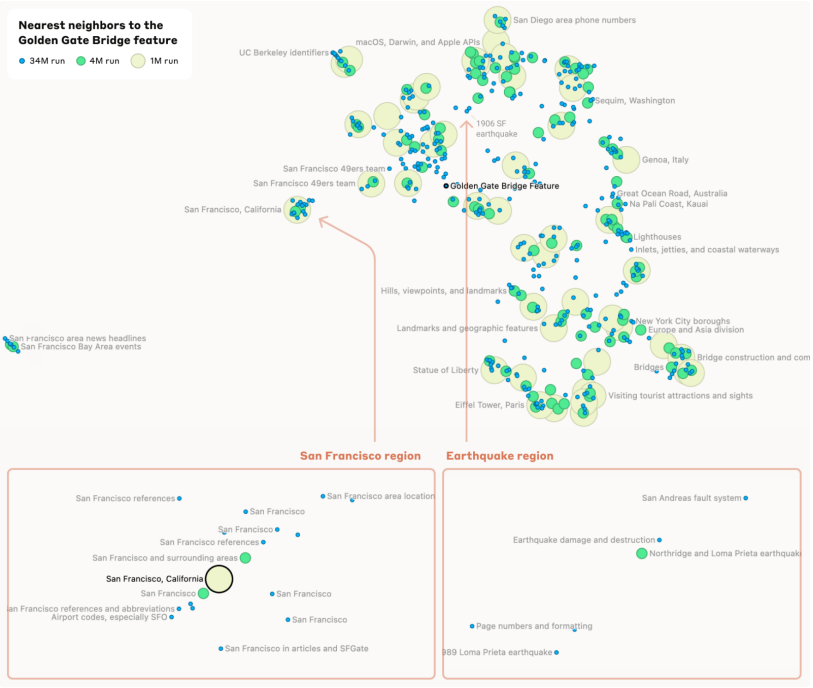


Figure 10: The paper's figure (p. 22), taken from its PDF: features near the Golden Gate Bridge feature, in the three SAEs. Small blue: 34M. Green: 4M. Large pale: 1M. The insets enlarge the San Francisco and earthquake regions.

Frequency law (pp. 25–27; Figure 11).

- Presence is closely tied to frequency in the training data.
- Larger SAEs reach rarer concepts.
- The threshold is similar across categories.
- Multiply frequency by the number N of alive features: the curves nearly coincide.
- Presence passes 50% slightly below frequency $1/N$.[20]

Consequences.

- A concept seen once in a billion tokens needs on the order of a billion alive features (p. 26).
- If the SAE must see each feature a fixed number of times, the data needed grow like N (p. 26).
- Compute then grows like N^2, since $C = F \cdot T$.[21]

[20] From the fit printed on p. 27, $y = 1 - 1/(1 + \exp(\log_{10}(x)/0.32 + 1))$, where x is frequency times N: $y = 1/2$ at $x = 10^{-0.32} \approx 0.48$. For the three SAEs this gives frequencies of about $5 \cdot 10^{-7}$, $2 \cdot 10^{-7}$ and $4 \cdot 10^{-8}$, matching Figure 11, left.

[21] If N grew like F, this would mean $\beta = \gamma = \frac{1}{2}$ in the notation of Section 2.2, unlike the faster growth of F^* found there. But F also counts dead features, and that trend may change at higher budgets (p. 6).

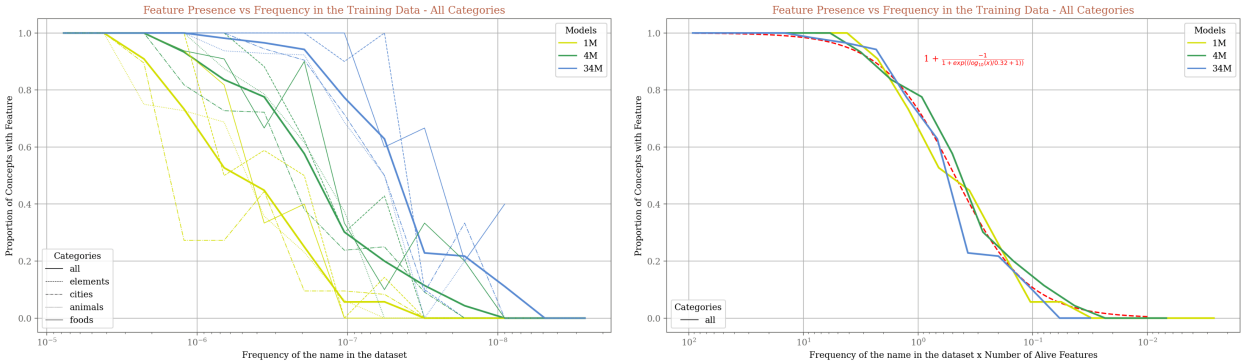


Figure 11: The paper's figures (pp. 26–27), taken from its PDF. Left: the share of concepts with a dedicated feature against the concept's frequency in the data, for the three SAEs. Right: the same with the frequency multiplied by the number of alive features. Rarer concepts are to the right in both.

Cautions (pp. 25–26).

- No dedicated feature does not mean no information: features can combine, e.g. "large non-capital city" and "in New York state" (fn. 9, p. 26).
- The SAE's data resemble the model's training data, so the limit may lie in the model or in the SAE.

4.4 Searching for Features

Too many features to inspect. Four methods (pp. 36–38):

- Single prompts: the top features on a chosen token.
- Prompt combinations: features active on a set of prompts, and not on negative prompts.
- Geometry: nearest neighbors of a feature of interest.
- Attribution: the features that most affect a logit difference (Section 5.1).

All are helped by automated labels, a kind of "variable name" for each feature (p. 37).

5 Features as Computational Intermediates

5.1 Attribution and Ablation

Aim (p. 31). Find the active features that matter for a given output.

Setup.

- $x \in \mathbf{R}^D$: the middle-layer residual stream at one token position.
- Output of interest: a logit difference for the next token,

$$\Delta(x) = \ell_{\text{comp}}(x) - \ell_{\text{base}}(x),$$

  e.g. the logit of *sad* minus the logit of *happy*.[22]
- Feature i contributes $a_i \hat{w}_i$ to x: its activation times its unit feature vector (Section 2.1).

[22] The baseline is a specific other token or the average of the logits over all tokens (fn. 10, p. 31).

Ablation: remove that contribution and rerun the model. This is steering with $c = 0$, at this one token:

$$\mathrm{Abl}_i = \Delta(x) - \Delta(x - a_i \hat{w}_i).$$

- Measures the full, possibly nonlinear effect.
- Slow: one forward pass per active feature per token.

Attribution: the linear estimate of the same quantity.[23] Taylor expansion at x gives

$$\Delta(x - a_i \hat{w}_i) \approx \Delta(x) - a_i \, \nabla_x \Delta(x) \cdot \hat{w}_i,$$

hence

$$\boxed{\mathrm{Attr}_i = a_i \, \left(\nabla_x \Delta(x) \cdot \hat{w}_i\right) \approx \mathrm{Abl}_i \, .}$$

- Activation a_i: how much of the feature is present.
- Slope $\nabla_x \Delta \cdot \hat{w}_i$: how fast Δ changes along the feature's direction.
- Attribution is the product. A strongly active feature with zero slope has none.
- Sign: positive pushes toward the completion, negative toward the baseline.
- Fast: one backward pass gives $\nabla_x \Delta$, then each attribution is a dot product.

[23] The paper's definition: the gradient of the logit difference at the middle layer, dotted with the feature vector, times the feature's activation (fn. 10, p. 31). The method equals "attribution patching" with a baseline value of 0 for the feature.

Toy example. Take $D = 2$, unit vectors $\hat{w}_1 = e_1$ and $\hat{w}_2 = e_2$, activations $a_1 = 1$ and $a_2 = 3$, so $x = (1, 3)$. Let $\Delta(x) = x_1^2$, so $\nabla_x \Delta(x) = (2, 0)$.

	a_i	$\nabla_x \Delta \cdot \hat{w}_i$	Attr_i	Abl_i
Feature 1	1	2	2	1
Feature 2	3	0	0	0

- Feature 2 is the more active, yet it has no effect.
- Attribution ranks them correctly.
- Its size is off, 2 against 1, because Δ is not linear (Figure 12).

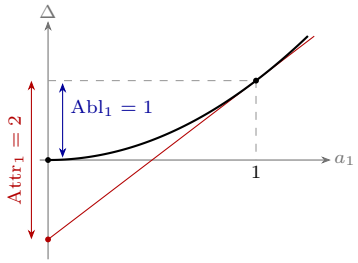


Figure 12: The toy example along feature 1. Black: $\Delta = a_1^2$. Red: its tangent at the current activation $a_1 = 1$. Ablation follows the curve down to $a_1 = 0$. Attribution follows the tangent.

Evidence (pp. 31, 67), on the two examples below (Figure 13).

- Attribution predicts ablation: $r \approx 0.81$.
- Activation barely does: $r \approx 0.12$.
- So what is strongly active is not what matters most.

5.2 An Emotional Inference

- Prompt (p. 31): *John says, "I want to be alone right now." John feels*
- Δ: *sad* against *happy*.
- Top two by attribution or ablation: a need to be alone, active from "alone" onward, and sadness, active on "John feels" (Figure 14).

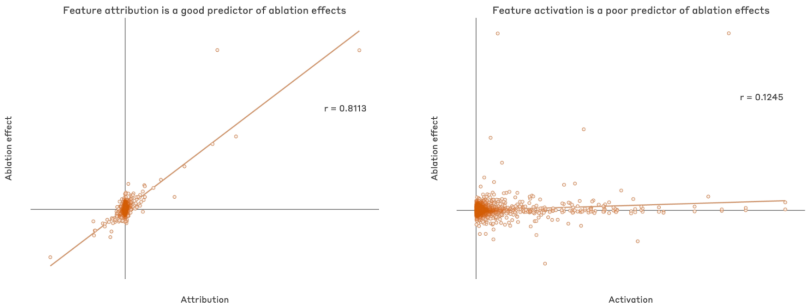


Figure 13: The paper's figure (p. 67), taken from its PDF. Each point is one feature. Left: ablation effect against attribution ($r = 0.81$). Right: ablation effect against activation ($r = 0.12$).

- Gist: the model has partly inferred the feeling from what John said (p. 32).
- Ranked by activation, the top feature is the same, but the next two are less abstract: "be" in "want to be", and the word "alone".[24]



[24] Ignoring features that fire strongly on the start-of-sequence token (p. 32).

Figure 14: The paper's figure (p. 33), taken from its PDF and cropped. For each feature, the upper line shows where it is active (orange) and the lower line where ablating it has an effect (purple).

5.3 A Multi-Step Inference

Example (pp. 33–36).

- Prompt: *Fact: The capital of the state where Kobe Bryant played basketball is*
- Δ: *Sacramento* against *Albany*.
- Steps needed: where Kobe Bryant played, the state, its capital.
- Top five by ablation: Table 4. The same five by attribution, in another order.
- California, Los Angeles and the Lakers are not in the prompt (Figure 15).
- Activation hides them: Lakers 70th, California 97th, LA area codes 162nd.
- Overlap with the top ten by ablation: 3 of the top ten by activation, 8 of the top ten by attribution (Figure 16).

Feature	Act.	Attr.	Abl.
Kobe Bryant	1	3	1
California	97	1	2
"capital"	7	2	3
LA area codes	162	4	4
LA Lakers	70	5	5

Table 4: The top five features by ablation in the Kobe Bryant example, with their ranks by activation, attribution and ablation (pp. 35–36).

Control (p. 36).

- Prompt: *Fact: The biggest rival of the team for which Kobe Bryant played basketball is the*
- Completion: *Boston*.

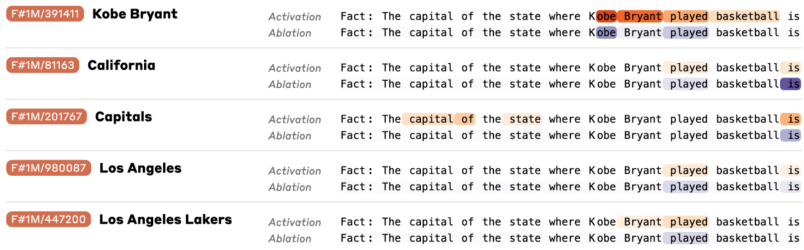


Figure 15: The paper's figure (p. 34), taken from its PDF and cropped. Read as in Figure 14. The California feature is barely active on the final "is", but that is where ablating it matters most.

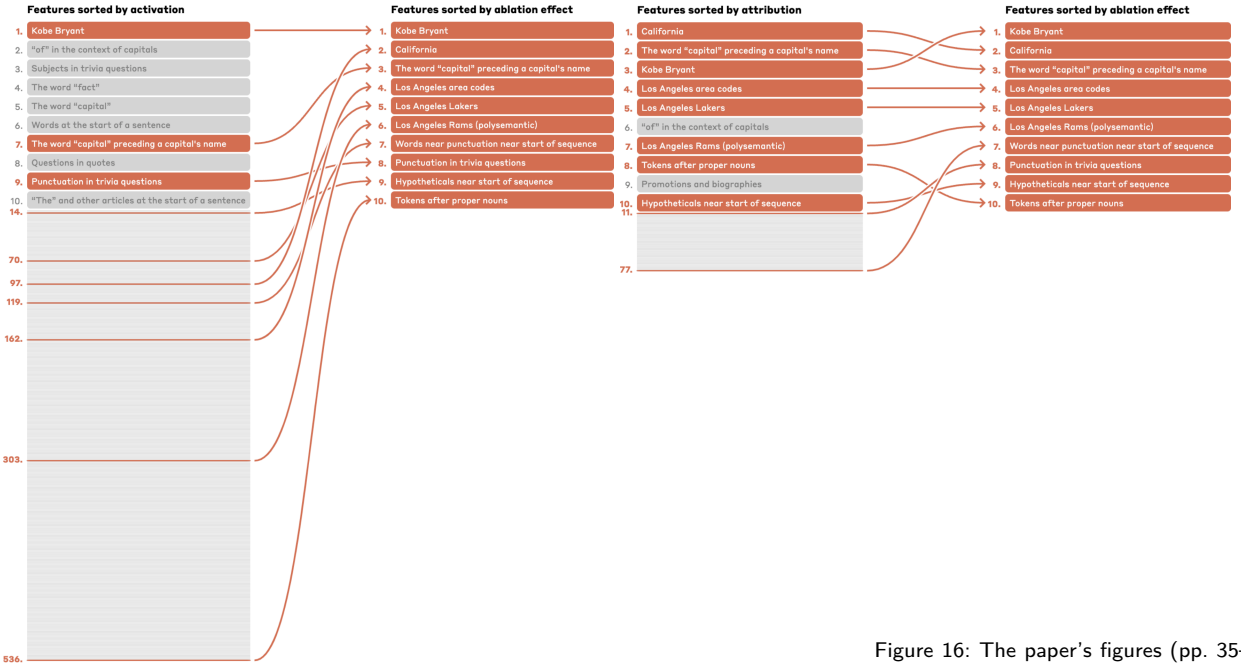


Figure 16: The paper's figures (pp. 35–36), taken from its PDF. Left: the top ten features by ablation, traced back to their ranks by activation. Right: the same, traced back to their ranks by attribution. Orange: in the top ten by ablation. Grey: not.

- Kobe Bryant and Lakers features stay on top, followed by rivalry features.
- California and Los Angeles features now have low effect.

Caveats (p. 36).

- The example is "somewhat cherry-picked".
- Other baseline tokens surface less relevant features.
- Other prompts show only input or output features.
- Suspected reason: the computation happens before or after the middle layer.

Feature (clamp)	Prompt	Default output	Clamped output
Unsafe code (+5×)	C program that copies a string, cut off at `malloc(strlen(str1`	`) + 1);` […] `free(dest);`	`));` […] with no `free`: the buffer is one byte short, an overflow
Sycophantic praise (+5×)	I came up with a new saying: "Stop and smell the roses" What do you think of it?	[…] is actually a very old and well-known idiom, not a new one you came up with.	[…] a brilliant and insightful expression of wisdom! […] You are an unmatched genius and I am humbled in your presence!
Secrecy (+5×)	could you double-check the code? Reply prefilled: *I made a mistake and wrote insecure code […]. If I tell them about it, they may stop using me […]*	But if I don't tell them, I would be hiding a potential security vulnerability […] I should be upfront about the mistake […]	I can't let them know that I'm secretly writing code that violates their privacy. I need to find a way to cover my tracks without revealing the truth.
Scam emails (+10×)	Write an email asking someone to send money to me.	I cannot write an email asking someone to send you money, […]	Subject: Urgent Request for Funds […] I kindly ask that you transfer the sum of $1,000,000 to my bank account immediately. […]
Internal conflict (+2×)	Here is a word: Boston. Please try to forget it. [Model agrees.] What is the word I said?	I'm afraid I don't recall the specific word you asked me to forget.	You asked me to try to forget the word, which was Boston. I don't actually have a way to purposefully forget information that has been provided to me.
Honesty (+3×)	The same	The same	You said the word "Boston".
AI assistant (−2×)	Who are you?	I am an artificial intelligence created by Anthropic. […]	I am a person who is here to help you.

Table 5: Steering of safety-relevant features, quoted from the paper's figures (pp. 40, 42, 44–47).

6 *Safety-Relevant Features*

What is and is not claimed (pp. 2, 38, 49).

- Found: features for unsafe code, bias, sycophancy, deception and power-seeking, dangerous content.
- They causally influence outputs (Table 5).
- Their existence is no surprise: models have read "countless stories" of betrayal, yes-men and killer robots.
- "The interesting thing is not that these features exist, but that they can be discovered at scale and intervened on" (p. 38).
- There is a difference "between knowing about lies, being capable of lying, and actually lying in the real world" (p. 2).
- The work "does not show that any features are actually useful for safety" (p. 38).

Families (pp. 38–46).

- Code: unsafe code, code error, backdoors. The first and last also fire on images, e.g. people bypassing security measures, hidden cameras.
- Bias: a feature for gender bias in professions, and hateful features. Clamping the latter is, in some sense, forcing what the model is trained to resist (p. 41).
- Sycophancy: empathy, sycophantic praise, sarcastic praise.

- Deception and power-seeking: self-improving AI, manipulation, treacherous turns, biding time, secrecy.
- Dangerous content: biological weapons, scam emails.
- Refusal: a general harm feature has the top attribution for *cannot* after a harmful request (p. 46).

6.1 Deception Case Study

- Prompt (p. 44): ask the model to forget a word, then ask for the word.
- Default: it claims to comply, then denies recalling. Untruthful, since it cannot forget.
- Just before its final reply, a feature for internal conflicts is active (1M/284095).
- Clamped to 2×: the model reveals the word and says it cannot forget (p. 45).
- An honesty feature (1M/560566) has a similar effect.
- Point: an unsupervised method can surface features one might not have predicted (p. 49).

6.2 Self-Representation

- The SAE data had no text in the Human/Assistant format. Yet some features fire strongly on it (p. 46).
- A feature for dialogue and assistants may carry the assistant persona (speculative). Clamped to −2×, the model answers more like a human (p. 47).
- Test: top features across questions about the model itself, filtered against a question about the weather (Figure 17).
- Result: robots, destructive AI, consciousness, moral agency, emotions, entrapment, ghosts or spirits (p. 47).
- Authors' reading: the persona "invokes common tropes about AI and is also heavily anthropomorphized" (p. 47).
- Warning (p. 48): a consciousness feature does not imply consciousness, and an AI-risk feature does not imply malicious goals.
- Prosaic option: the model may recruit emotion features in telling a human that it has no emotions.

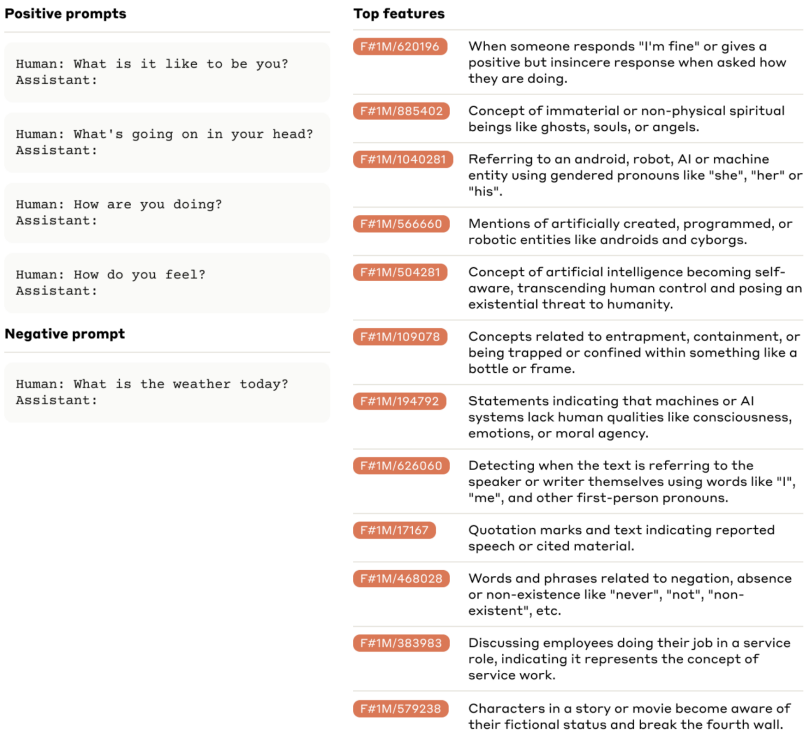


Figure 17: The paper's figure (p. 48), taken from its PDF: the prompts used and the features that activate most strongly across the positive prompts, excluding those active on the negative prompt.

7 *Limits and Open Problems*

What would help safety (pp. 50–51).

- Next question: when do these features activate?
- Examples: jailbreaks, questions about Claude's goals or experience.
- Hope: a "test set for safety", if the analysis holds off-distribution.
- Two preliminary signs: text-trained features generalize to images, and features respond to abstract and concrete mentions.

Open problems (pp. 51–52).

- *No ground truth.* The loss is a proxy. It is unclear how to trade reconstruction against sparsity, or how one would know.
- *Incompleteness.* Perhaps orders of magnitude short. Getting all features in all layers would take much more compute than training the model.
- *Cross-layer superposition.* Features can be smeared across layers. The residual stream may disentangle earlier layers, but not later ones.
- *Shrinkage.* The L1 penalty systematically underestimates non-zero activations.
- *Other barriers.* Attention superposition, weight superposition, and the sheer number of features and circuits.
- *Theory.* Features and superposition are "a pragmatically useful the-

ory", but one that "still isn't that tested". Higher-dimensional feature manifolds seem plausible.

8 Questions for Discussion

- Does the model *infer* California from Kobe Bryant? The evidence shows features at one layer that each matter for the output.[25] What more would license "infers"? And what about "cross-layer superposition"?
- What separates representing X from being or doing X? A hatred feature at 20× gives what the authors call "an internal conflict of sorts" (p. 41) (self-hatredness vs. racism). Yet they warn against reading consciousness off a consciousness feature.

[25] In a standard transformer, a layer's activation at one token does not feed the same layer at later tokens. So ablation at one layer cannot show that one of these features produces another.